UQ-QNN Repository Overview, Development Progress, and Rerun Experiment Analysis

Project Presentation Notes

April 2, 2026

Contents

1 Repository Content and Goal

The repository `uq-qnn` implements a framework for *uncertainty-aware photonic quantum neural networks*. The core goal is to train photonic circuits such as Clements meshes and memristive photonic variants on regression and classification tasks while preserving a practical workflow for:

- differentiable training via the parameter-shift rule (PSR),
- reproducible experiment execution and reporting,
- support for both fast analytical simulation and full Perceval-based simulation,
- uncertainty quantification through repeated noisy forward passes,
- controlled comparisons between standard photonic circuits and memristive circuits.

At a high level, the data flow is:

$$x \to \phi(x) = 2\arccos(x) \to \text{photonic circuit} \to \text{Born-rule probabilities} \to \text{loss} \to \text{PSR gradients}.$$

The repository now contains three complementary layers:

1. **Modeling and simulation**: Clements circuits, memristive feedback, singles/coincidence measurements, NumPy and Perceval backends.
2. **Training and experimentation**: `SimConfig`, `Experiment`, uncertainty analysis, metrics, artifact export.
3. **Analysis and communication**: reproducible report folders, comparison scripts, benchmark scripts, and now this presentation material.

2 Project Stages and Progress

The changelog shows a clear staged evolution from a collection of scripts toward a structured research codebase.

2.1 Stage 1: Baseline simulation and PSR training

The original codebase established the photonic training loop: circuit simulation, Born-rule outputs, and PSR-driven optimization. This stage proved that Clements-style photonic networks could be trained on synthetic tasks, but the workflow was still script-heavy and difficult to compare across runs.

2.2 Stage 2: Experiment API and reproducibility

A major step was the introduction of the `Experiment` API and explicit script-local configuration. This changed the repository from a collection of ad hoc scripts into a reproducible framework with:

- timestamped run folders,
- saved metrics and artifacts,
- centralized logging,
- a consistent mapping from configuration to training and inference.

This stage directly advanced the goal of making uncertainty-aware photonic experiments repeatable and comparable.

2.3 Stage 3: Hardware abstraction and cleaned interfaces

The next stage introduced hardware profiles and a cleaner separation of concerns. With this, the repository could represent ideal and noisy hardware conditions without rewriting experiment scripts. In parallel, dead legacy utilities and configuration indirection were removed. This moved the project closer to realistic photonic deployment scenarios.

2.4 Stage 4: PhotonicCircuit facade and package cleanup

The addition of `PhotonicCircuit`, public circuit helpers, and a simulation package reorganization improved code clarity and reduced duplicated logic. This stage made the fast analytical backend easier to use directly and made the architecture more maintainable.

2.5 Stage 5: Correctness and developer quality

Further work tightened typing, fixed edge cases in hardware noise and memristive uncertainty handling, and added regression tests. This stage matters because the research questions in the repository are sensitive to subtle simulator and gradient issues.

2.6 Stage 6: Fast memristive NumPy backend and targeted reruns

The latest work targeted a key bottleneck: memristive NumPy simulations were correct but slow because they rebuilt full unitaries at every step. We introduced an optimized state-propagation path for memristive singles runs, first for discrete mode and then for swipe mode. To evaluate both the engineering change and the modeling consequences, we reran and added several experiments focused on memristive circuits.

3 What We Changed to Reach the Goals

The repository goals were not reached through one single feature but through a sequence of structural and experimental changes.

3.1 Workflow changes

- Replaced hidden global configuration with explicit per-script `CONFIG` dictionaries.
- Added the `Experiment` context manager to standardize training, prediction, uncertainty, logging, and artifact saving.
- Moved results into timestamped report folders with `run_summary.json` to support later analysis and presentation.

3.2 Architecture and simulator changes

- Promoted public circuit helpers such as memristive index normalization and phase-to-MZI mapping.
- Added `PhotonicCircuit` as a lightweight interface for fast NumPy simulations.
- Refactored simulation code into a clearer package with a dedicated memristive state manager.

3.3 Correctness and robustness changes

- Fixed classification and scalar-noise edge cases.
- Corrected gradient handling for memristive weights.
- Added more tests to protect against behavioral regressions.

3.4 Performance changes

The most recent change is especially important for the current research direction:

- a dedicated fast NumPy memristive singles path now propagates a single-photon state vector through the Clements mesh instead of rebuilding a full unitary at each step,
- this path supports both discrete and swipe-mode memristive runs,
- targeted regression tests verify parity with the previous slower loop.

This performance work was necessary because investigating memristor placement and multi-memristor configurations becomes much more expensive if every rerun must pay the old simulation cost.

4 Rerun Experiments Considered in This Document

This document focuses on the reruns and new experiments created during the latest memristor investigation:

1. **Function and memristor comparison**: standard vs. memristive architecture across several synthetic functions.
2. **Smooth-step placement comparison**: one memristor placed at several different phase positions.
3. **Smooth-step multi-memristor comparison**: no memristor, one memristor, or two memristors.
4. **Memristive backend benchmark**: fast NumPy memristive path vs. the legacy reference loop.

The corresponding report folders are:

- `reports/function_and_memristor_comparison/2026-04-01_212859/`
- `reports/smooth_step_memristor_placement_comparison/2026-04-01_223609/`
- `reports/smooth_step_multi_memristor_comparison/2026-04-01_230245/`
- `reports/memristive_backend_benchmark/2026-04-01_215523/`

5 Experiment 1: Function and Memristor Comparison

This rerun compares a standard 6-mode Clements mesh to a 6-mode memristive variant with one memristive phase. The goal was to answer a basic modeling question: *does the memristor help function fitting across representative regression targets?*

5.1 Selected results

Function	Standard RMSE	Memristor RMSE	Standard R^2	Memristor R^2
Quartic	0.0200	0.0342	0.9876	0.9637
Sinusoid	0.0274	0.0210	0.9455	0.9680
Multi-modal	0.1261	0.1236	0.2757	0.3043
Smooth step	0.0714	0.0754	0.8393	0.8209

Table 1: Main fit metrics from the rerun function-vs-memristor comparison.

5.2 Analysis

The memristor does *not* provide a uniform advantage.

- It helps on the sinusoid and slightly improves the multi-modal case in terms of RMSE/R^2.
- It hurts on quartic and smooth step.
- Calibration behavior remains unstable; for example the smooth-step memristor run shows substantially worse NLL and very poor 95% coverage.

This experiment led to the next question: perhaps the issue is not memristive feedback itself, but *where* the memristor is placed in the 6-mode mesh.

Figure 1: Metrics from the function-and-memristor rerun.

6 Experiment 2: Smooth-Step Memristor Placement Comparison

The smooth-step function is a useful focused test because the earlier broad comparison suggested that a single memristor was not clearly helping on this task. We therefore reran the smooth-step problem while moving the memristor to different positions.

6.1 Placements tested

The following configurations were tested:

- no memristor (standard baseline),
- phase 0 on MZI(0,1),
- phase 2 on MZI(2,3),
- phase 4 on MZI(4,5),
- phase 6 on MZI(1,2),
- phase 8 on MZI(3,4).

6.2 Key metrics

Placement	RMSE	R^2	Coverage	NLL
Standard	0.0718	0.8377	0.368	3.790
Phase 0	0.4038	-4.1344	0.002	90936.430
Phase 2	0.0712	0.8403	0.278	3.766
Phase 4	0.0712	0.8403	0.278	3.766
Phase 6	0.0757	0.8193	0.048	14.559
Phase 8	0.0712	0.8403	0.278	3.766

Table 2: Smooth-step placement rerun metrics.

6.3 Analysis

This rerun gives a nuanced answer.

- The memristor **placement matters strongly**. Phase 0 is catastrophic and destabilizes the fit.
- Phases 2, 4, and 8 produce almost the same fit quality and are only marginally better than the standard baseline in RMSE/R^2.
- Phase 6 is worse than the standard baseline and dramatically worse in uncertainty coverage and NLL.

The most important conclusion is that the repository's memristive mechanism is not "plug-and-play." A memristor can degrade the model if it is placed too close to the data encoding or in a part of the mesh where the feedback creates an unfavorable recurrent bias.

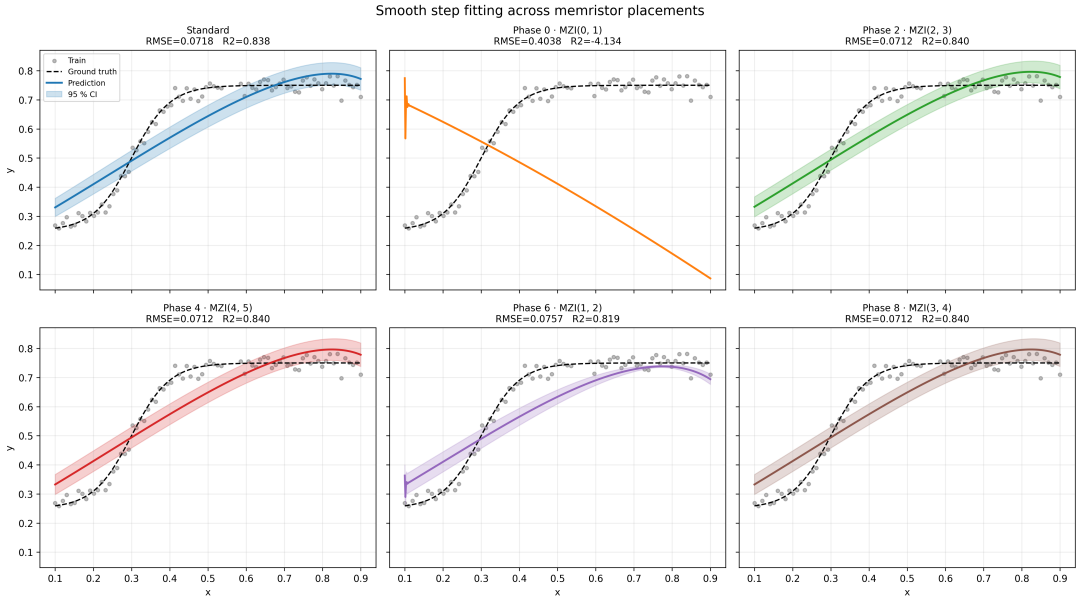

Figure 2: Smooth-step fits for different memristor placements.

7 Experiment 3: Smooth-Step Single vs. Multiple Memristors

After the single-placement rerun, we tested whether adding *more than one* memristor could recover a useful effect. The chosen pair was phases 6 and 8, corresponding to the MZI(1,2) and MZI(3,4) positions.

7.1 Configurations

- standard baseline,
- single memristor at phase 6,
- single memristor at phase 8,
- two memristors at phases 6 and 8.

7.2 Results

Configuration	RMSE	R^2	Coverage	NLL
Standard	0.0718	0.8377	0.368	3.790
Single 6	0.0757	0.8193	0.048	14.559
Single 8	0.0712	0.8403	0.278	3.766
Double 6+8	0.0764	0.8164	0.052	12.595

Table 3: Smooth-step single-vs-multiple memristor rerun metrics.

7.3 Analysis

The two-memristor configuration did *not* improve the smooth-step task.

- The best RMSE in this group comes from the standard baseline or the single memristor at phase 8, depending on whether one prioritizes accuracy or calibration.
- Adding the second memristor at phase 6 degrades the model relative to single phase 8.
- This suggests that the phase-6 feedback pathway is harmful in this task and remains harmful even when paired with another memristor.

So, the correct takeaway is not simply "more memristors are better." Instead, memristive feedback seems highly task- and placement-dependent.

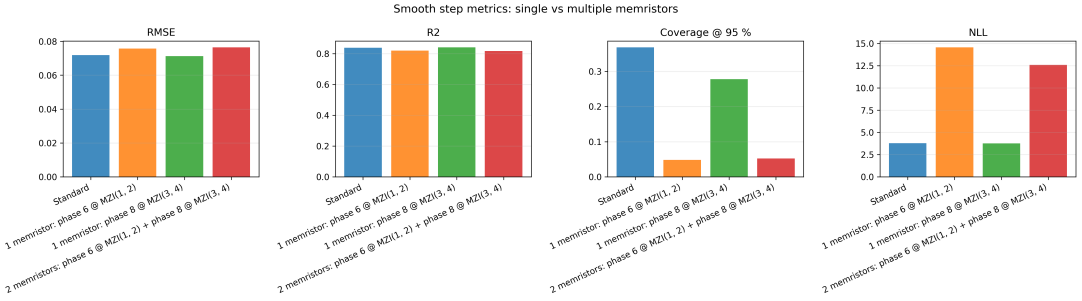

Figure 3: Smooth-step comparison for one vs. two memristors.

8 Experiment 4: Memristive Backend Benchmark

The repository work was not only about modeling; it also targeted simulator efficiency. We benchmarked the new fast NumPy memristive backend against the previous reference implementation.

Case	Fast [ms]	Legacy [ms]	Speedup	Notes
Discrete	5.562	22.994	4.13x	one memristor, no swipe
Swipe	23.315	109.183	4.68x	memristive swipe mode
Swipe inline	22.744	100.722	4.43x	inline encoding + swipe

Table 4: Benchmark of the fast memristive NumPy backend.

8.1 Analysis

The performance change is meaningful for research iteration.

- The new backend yields roughly a 4.1x–4.7x runtime reduction on the tested workloads.
- The gain is especially important for swipe mode, where repeated sub-evaluations per data point make the old implementation expensive.
- Because parity tests were added, this acceleration does not trade away correctness.

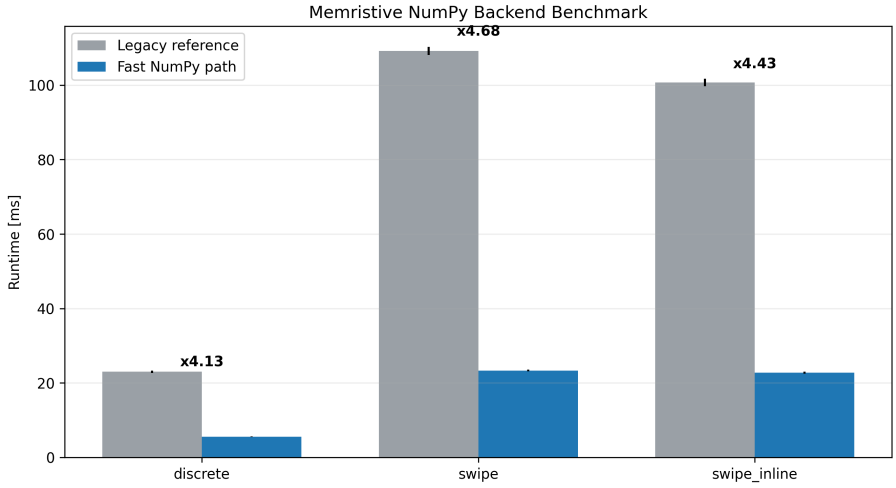

Figure 4: Benchmark plot for the new memristive NumPy backend.

9 Overall Conclusions from the Reruns

The rerun experiments give a balanced picture of the current state of the repository.

9.1 What worked

- The repository now supports reproducible comparisons and benchmarking with minimal boilerplate.
- The new memristive NumPy backend significantly accelerates the practical workflow.
- Memristive feedback can improve some functions, especially periodic structure such as the sinusoid.

9.2 What did not work as hoped

- A single memristor is not universally beneficial.
- On smooth-step fitting, most memristor placements do not beat the standard baseline in a meaningful way.
- Two memristors at phases 6 and 8 do not improve the smooth-step task; in fact, they hurt relative to the best single-placement variant.

9.3 Interpretation

The engineering progress is clear: the framework is more modular, faster, better tested, and easier to analyze. The scientific result is more subtle: memristive photonic feedback does not automatically improve function fitting. Its value depends on task, placement, and uncertainty behavior. That is still a useful result, because it narrows the search space for future memristor studies.

10 Recommended Next Steps

Based on the reruns, the next stage should focus on *structured exploration* rather than simply adding more memristors.

1. Test two-memristor pairs that avoid phase 6, since that position repeatedly produced poor results on smooth step.
2. Extend the comparisons to harder targets where recurrence may be more naturally beneficial than on a simple smooth step.
3. Study memory depth explicitly; placement and depth may interact.
4. Add benchmark cases with multiple memristors so performance scaling is quantified alongside modeling quality.
5. Continue using calibration metrics, not just RMSE/R^2, because some memristive runs appear accurate but poorly calibrated.

11 Build Notes

This report can be compiled from the repository root with:

```
cd presentation
pdflatex repository_overview.tex
```

The accompanying Beamer slide deck is stored as `rerun_experiments_slides.tex` in the same folder.